

MEDIATEK

MT8676 Hypervisor Suspend & Resume User Manual

版本: 1.3
出版日期: 2025-07-11

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-10-28	郑孝俊	正式版
1.1	2024-11-13	郑孝俊	更新 L+L+A
1.2	2025-02-21	陈杰	更新 Modem 和 RTC 唤醒
1.3	2025-07-11	陈杰	更新 Android STR Debug 方式 更新如何分析休眠功耗大的问题

目录

版本记录 2

目录 3

图片目录 3

表格目录 3

1 Suspend/Resume 4

 1.1 概述 4

 1.2 架构/流程概要 4

 1.2.1 Hypervisor Suspend 流程 4

 1.2.2 Hypervisor Resume 流程 6

 1.2.3 RTC 或 Modem 唤醒流程 7

 1.3 常见问题/故障排除 8

 1.3.1 如何判定系统休眠成功 8

 1.3.2 如何确认唤醒源 8

 1.3.3 唤醒源梳理 9

 1.3.4 如何分析不能休眠的问题 10

 1.3.5 如何分析休眠功耗大的问题 11

 1.3.6 如何分析 Android uos 无法 Suspend 问题 11

 1.3.7 Hypervisor Debug 命令 11

附件一 附加条款 13

图片目录

图 1-1. Hypervisor suspend 流程 5

图 1-2. Hypervisor resume 流程 6

图 1-3. Wakelock dump 示意图 10

表格目录

表 1-1. MT8676 唤醒源列表 9

1 Suspend/Resume

1.1 概述

本章节主要介绍 MT8676 Hypervisor Suspend/Resume 流程以及常见问题的处理方法。

在 MT8676 平台上，Suspend 即 Suspend to RAM，此状态使所有的设备进入低功耗状态，仅保留 RAM 自刷新。

Suspend Power State：

- Tasks “freezed”
- 外部设备/内部部分模块关电或者进入低功耗模式
- System PLL/clock close
- ARM off
- DRAM 自刷新
- PMIC 进入低功耗模式
- VCORE off
- SPM run
- 等待硬件唤醒事件

在 MT8676 Hypervisor (L+L+A) 系统上，Host OS SoS 与 Guest OS Tbox 系统都是 Yocto，Guest OS IVI 系统是 Android。系统级 Suspend 由 Host OS 发起，触发 Guest OS 运行 Suspend Flow，Guest OS 都完成 Suspend Flow 之后，Host OS 可以进入 Suspend，Host OS Suspend 完成之后整个系统才能进入省电的深度休眠模式。

1.2 架构/流程概要

1.2.1 Hypervisor Suspend 流程

Hypervisor Suspend 框架图如下（以 EINT 触发系统进出休眠为例）：

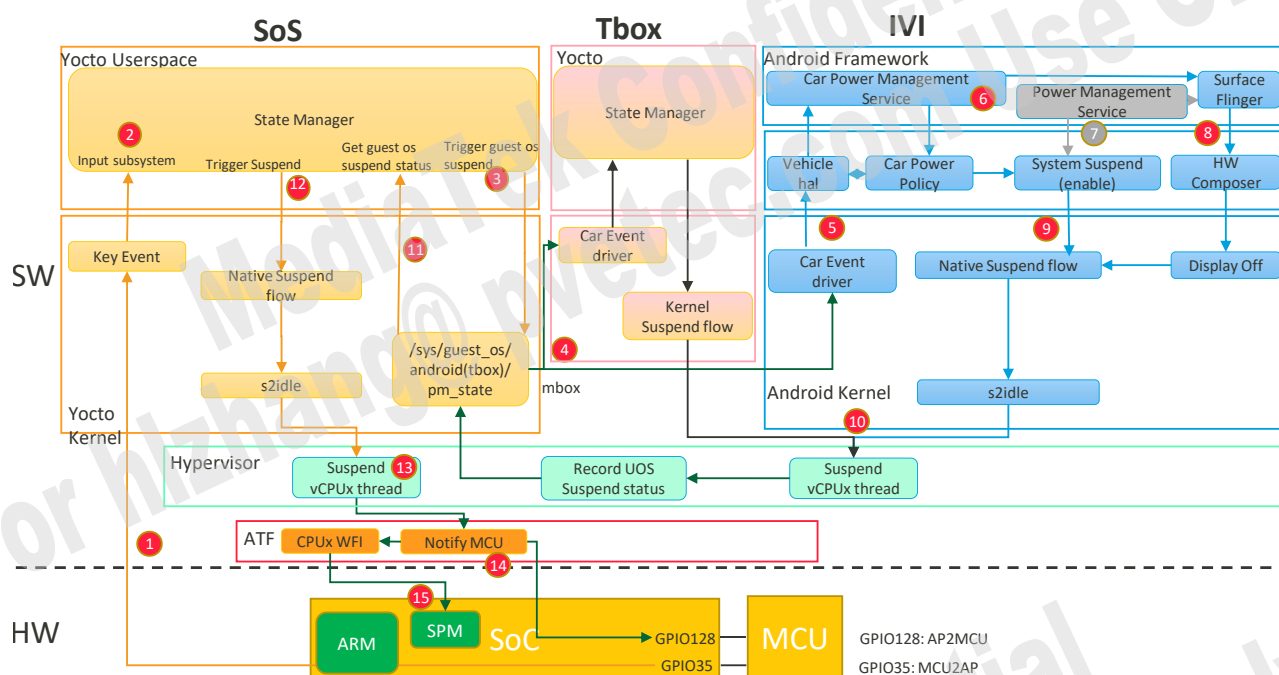


图 1-1. Hypervisor suspend 流程

主要的流程说明:

1. 平台进出休眠主要由 MCU 发起，MCU 通过 GPIO35 (MCU2AP) 的状态控制 SoC 进出休眠，GPIO35 被拉低后，会触发中断，该中断会被 Hypervisor 分发给 SoS 系统处理，SoS Kernel 收到中断后，将 Key Event 上报。
2. 在 SoS UserSpace 中，MediaTek 添加了一段控制逻辑 SleepManager (State Manager)，用于管理 Guest OS 的电源状态，这层逻辑主要是 SoC 通过 Input Sub System 获取 Key Event 来触发的。
3. 当检测到休眠事件后，SoS UserSpace 写 powerkey event 到 `/sys/guest_os/android(tbox)/pm_state`，向两个 Guest OS (Android/Yocto) 发起休眠请求。
4. SoS 的 Kernel Driver 通过 Mailbox 的方式将 Suspend 请求通知到两个 Guest OS 的 Car Event Driver。
5. 在 Car Event Driver 中，上报虚拟 Power Key (Keyevent=87)，触发 Guest OS 端完整的休眠流程。
6. Guest OS IVI 由于 Vehicle HAL 没有现成的接口对接 Car EVENT Driver，所以这里增加 Input sub system 支持，等待 Kernel 端送来的虚拟 Power Key，VHAL 收到后会通知 Car Power Management Service 启动休眠流程。
7. Power Management Service 依然监听 Power Key Event (Keyevent=116)，这里没有执行 PMS Flow。
8. 触发关闭显示相关硬件。
9. CPMS 在没有唤醒锁检查的情况下触发 Suspend Flow，触发 Android Kernel Suspend。Kernel Suspend Flow 与原生系统中的流程相同。根据执行顺序，它依次调用设备注册的 `Prepare/Suspend/Suspend_late` 回调函数，在休眠阶段为每个 Device driver 完成必要的准备工作。

10. 各 Device 休眠后, 送 `s2idle` SMC 指令到 ATF 去关闭 CPU core, 指令会被 Hypervisor 拦截, 并挂起对应的 vCPU 线程。Android 的休眠完成, Hypervisor 会记录的 Android 状态将从 running 改为 suspend, 方便 Host OS 通过节点查询。
11. Host OS 端触发 Guest OS 端休眠后, 会阻塞并等待, 直到从节点 `/sys/guest_os/android(tbox)/pm_state` 获取到两个 Guest OS 都为 Suspend 的状态。
12. 待 Guest OS 都 Suspend 后, Host OS 端上报 key code 87 触发关闭显示相关, 然后再检查 Wakelock 状态, 如无锁, 则触发 Host OS Kernel Suspend Flow (`echo mem > /sys/power/state`)。
13. 执行 Host OS Kernel Suspend, 完成 Device Suspend 后, 送 `s2idle` SMC 指令到 ATF 去关闭 CPU core, 指令会被 Hypervisor 拦截, 并挂起对应的 vCPU 线程, 然后虚拟机做好 backup, Hypervisor 将 `s2idle` 指令再发送给 ATF, 将物理 CPU 真正关闭。
14. 在 ATF 中, CPU0 执行 WFI 指令前, 先将 GPIO128 (AP2MCU)拉低, 通知 MCU, SoC 端已经完成 Host OS 和 Guest OS 的 Suspend Flow。
15. 待 Arm 相关的硬件被关闭后, SPM (System Power Manage, 用于电源管理的 MCU) 将接管整个系统的资源, 控制 DRAM 进入自刷新模式, 关闭 26M 和 VCORE, 通知 PMIC 进入 Low Power Mode。至此, 整个休眠流程完成, 系统处于低功耗状态, 等待唤醒事件发生。

1.2.2 Hypervisor Resume 流程

Resume Flow 与 Suspend 相反, 可以参考图 1-2:

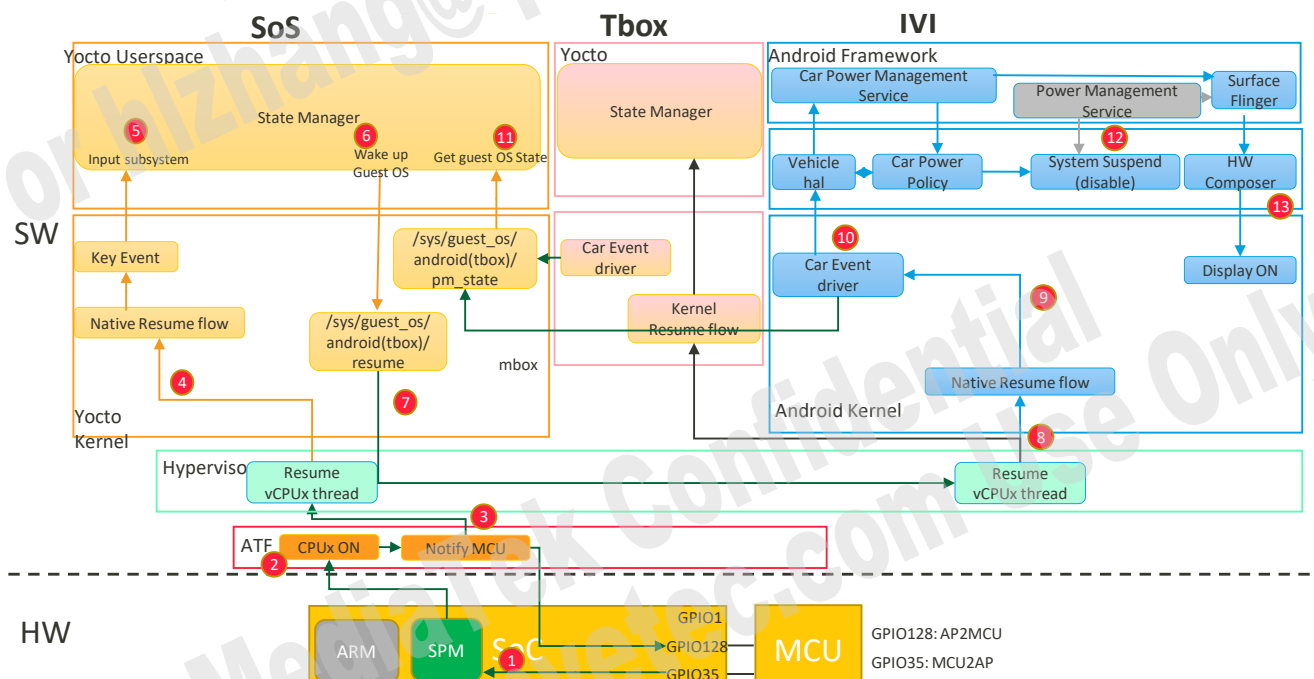


图 1-2. Hypervisor resume 流程

主要流程：

1. MCU 收到唤醒事件后，拉高 GPIO35 (MCU2AP)，SPM 收到 EINT 后，依次打开释放的资源，如通知 PMIC 离开 Low Power Mode，打开 VCORE 和 26M，发送命令通知 DRAM 离开自刷新。
2. 给 CPU0 上电，执行 CPU Resume 流程，再跳到 Suspend 前设定的地址，开始执行系统 Resume 流程。
3. 拉高 GPIO128，将 SoC 已经醒来的消息发送给 MCU。(可根据需求调整通知 MCU 的时间点，如等 Android Resume 完成再拉高 GPIO128)。
4. Hypervisor 中 vCPUx 线程恢复后，跳到 Host OS Linux Kernel，再执行 Native Kernel Resume Flow，将唤醒事件通过 Key Event 上报。
5. SoS 的 SleepManager 通过 Input Subsystem 上报的事件判断后，将决定再次休眠或者继续唤醒 Flow，像 MCU 拉高 GPIO35 这种就会继续唤醒 Flow。
6. 如果是可以唤醒整个系统的 Key Event，则上报 keyevent 87 去 resume Weston，并 hold wakelock，防止再次休眠。
7. 写节点 `/sys/guest_os/android(tbox)/resume`，触发 Guest OS 唤醒。
8. Hypervisor 收到后，恢复 vCPU0 线程。
9. Enable Non-Boot CPUs，恢复 vCPU1~7 线程，开始 IVI 及 Tbox Kernel Resume Flow。
10. IVI 基于 CPMS 架构，无需判断 keyevent，默认所有事件都可以触发整个系统 Resume。
11. 在 Host OS 端，可以通过节点 `/sys/guest_os/android(tbox)/pm_state`，使用 Mailbox 机制查询两个 Guest OS 系统 (Android/Tbox) 状态，如果 Guest OS 端 kernel Resume 完成则更新状态为 Running。
12. Android 端，Car Power Management Service 收到唤醒事件后，Disable System Suspend，防止误触发休眠。Tbox 端，由 SleepManager 主动 hold wakelock。
13. 通知打开显示相关模块，至此，整个唤醒流程完成，可以开始人机交互。

1.2.3 RTC 或 Modem 唤醒流程

RTC 或 Modem，从实际场景以及节省功耗的角度来说，它们唤醒的流程有所不同。

参考图 1-2，SPM 收到 RTC 或 Modem 唤醒后，1-6 的流程没有变化。第 7 步，SOS 的 SleepManager 通过 Key Event 的 Device Name，判断出是 RTC 或 Modem 触发的唤醒，则根据实际需求做不同的唤醒逻辑，例如仅需要唤醒 Tbox OS。Tbox Resume 后，需要 hold wake_lock 再处理任务，处理完任务后 release wake_lock。而 Host OS 端，当触发 Tbox Resume 后，会持一个临时的 wake_lock，通过节点 `/sys/guest_os/tbox/pm_state`，使用 Mailbox 机制查询 Tbox OS 的状态，如果 Tbox 端已完成任务并重新休眠，Host OS 端则会释放临时的 wake_lock。最后，SOS 端监测到没有 hold wakelock，会重新休眠。

1.3 常见问题/故障排除

1.3.1 如何判定系统休眠成功

屏幕熄灭可能仅表示系统进入浅睡眠，这并不代表系统已经成功进入休眠状态。

判定系统是否休眠成功需要查看 Kernel log，若系统成功进入 Suspend，则 Host OS 端不再打印 Kernel log。

如果需要通过 log 判断休眠是否成功，需要先下这几行命令打开更多的 debug log 信息：

```
adb shell "echo 8 8 8 8 > /proc/sys/kernel/printk"
adb shell "echo 1 > /sys/module/kernel/parameters/initcall_debug"
adb shell "echo 1 > /proc/mtprintk"
```

Kernel log 中，关键字“PM: Syncing filesystems ...”表示 Kernel 开始执行 suspend 流程。

Kernel log 中，关键字“suspend of devices complete after xxx msecs”表示 device suspend 完成。

Kernel log 中，关键字“late suspend of devices complete after xxx msecs”表示 device late suspend 完成。

Kernel log 中，关键字“noirq suspend of devices complete after xxx msecs”表示 device noirq suspend 完成。

Kernel log 中，关键字“suspend enter”表示 suspend 流程完成，系统已进入 suspend 状态。

1.3.2 如何确认唤醒源

在 Kernel log 中搜索关键字“suspend wake up by”，可以查看唤醒源。

如：

Pwrkey 唤醒（及其他 EINT 唤醒）：

```
[SPM] suspend wake up by R12_EINT_EVENT_B, timer_out = 207308
```

如果要看具体是由哪个 EINT 唤醒，需先从日志查看：

```
EINT xxx is pending
```

再通过 cat /proc/interrupts 看 xxx 对应的是哪个中断。

Modem 相关唤醒：

```
[SPM] suspend wake up by R12_CCIF0_EVENT_B, timer_out = 1825253
```

定时器 (PCM_Timer) 唤醒:

[SPM] suspend wake up by PCM_TIMER, timer_out = 65612

1.3.3 唤醒源梳理

目前 MT8676 支持的唤醒源列表如下:

表 1-1. MT8676 唤醒源列表

Name	Control Bit	Description	Can It be Disabled
R12_PCM_TIMER	0	设定计时器，超时后唤醒系统	Y
R12_KP_IRQ_B	2	检测到键盘按下/释放动作	Y
R12_APWDT_EVENT_B	3	RGU 在 suspend mode 中唤醒 CPU	Y
R12_APXGPT1_EVENT_B	4	AP GPT 计时器超时事件	Y
R12_CONN2AP_SPM_WAKEUP_B	5	Connectivity IC (Wi-Fi/BT/GPS) 唤醒系统	Y
R12_EINT_EVENT_B	6	EINT 事件唤醒系统	N
R12_CONN_WDT_IRQ_B	7	Connectivity IC watchdog timeout 唤醒系统	Y
R12_CCIF0_EVENT_B	8	MD to AP CCIF 唤醒事件	Y
R12_CCIF1_EVENT_B	9	MD to AP CCIF 唤醒事件	Y
R12_SSPM2SPM_WAKEUP_B	10	SSPM 相关唤醒系统	Y
R12_SCP2SPM_WAKEUP_B	11	SCP sensor 相关唤醒系统	N
R12_ADSP2SPM_WAKEUP_B	12	ADSP 相关唤醒系统	Y
R12_USBX_CDSC_B	14	USB 相关唤醒系统	Y
R12_USBX_POWERDOWN_B	15	USB 远程唤醒	Y
R12_SYS_TIMER_EVENT_B	18	System timer 唤醒系统	N
R12_EINT_EVENT_SECURE_B	19	EINT event 唤醒系统	N
R12_SCP_CIRQ_IRQ_B	22	SCP_CIRQ 唤醒事件	Y
R12_MD2AP_PEER_EVENT_B	23	MD 相关唤醒系统	Y
R12_MD1_WDT_B	25	MD1 看门狗超时	Y
R12_REG_CPU_WAKEUP_B	28	内部唤醒源	N
R12_APUSYS_WAKE_HOST_B	29	APUSYS 的唤醒源	Y
R12_PCIE_WAKEUP_EVENT_B	30	PCIe 相关唤醒源	Y

Name	Control Bit	Description	Can It be Disabled
R12_MSDC_WAKEUP_EVENT_B	31	MSDC 相关唤醒源	Y

您可以按照以下方式禁用列表中可以禁用的唤醒源。但是，请注意，只有在“Can It be Disabled”列中标注为可禁用（Y）的唤醒源才能被禁用：

使用系统提供的调试命令可以快速禁用特定的唤醒源：

1. 使用如下命令读取当前系统唤醒源设定：

```
cat /proc/mtk_lpm/power/suspend_ctrl | grep -i wake_src
```

2. 在表 1-1 的“Control Bit”栏位中找到您想要禁用唤醒源所对应的 control bit 位，然后将读出的 wake_src 的对应 bit 写 0，最后用下述命令将修改后的 wake_src 写进系统（不需要禁用的唤醒源的 control bit 位不进行修改）：

```
echo wake_src 0x***** > /proc/mtk_lpm/power/suspend_ctrl
```

3. 如果想要再启用某个唤醒源，就将对应的 control bit 再写 1 即可。

1.3.4 如何分析不能休眠的问题

如系统灭屏后无法进入休眠，可以用以下命令判定是哪个 wakelock 阻止系统进入休眠：

- 查看 Kernel 中有哪些模块持锁：

```
cat /sys/kernel/debug/wakeup_sources
```

观察输出结果的第 5 列 active_since，数字不为 0 且一直在增大的就是阻止系统进入待机的 wakelock。

例如图 1-3 中就是 USB 阻止了系统进入待机状态：

```
spm8675p1_64:/ # cat /sys/kernel/debug/wakeup_sources
```

name	active_count	event_count	wakeup_count	expire_count	active_since	total_time	max_time	last_change	prevent_suspend_time
event2	0	0	0	0	0	0	0	0	0
event1	8	8	0	0	0	203	145	41989	0
event3	0	0	0	0	0	0	0	0	0
event0	0	0	0	0	0	0	0	0	0
inotify	20	20	0	0	0	4	1	130586	0
eventpoll	2	2	0	0	0	0	0	31582	0
[timerfd]	30	30	0	0	0	0	0	130596	0
[timerfd]	18	18	9	0	0	1223	151	41937	0
[timerfd]	0	0	0	0	0	0	0	0	0
[timerfd]	18	18	4	0	0	575	151	121306	0
[timerfd]	20	20	2	0	0	284	131	37344	0
[timerfd]	0	0	0	0	0	0	0	0	0
[timerfd]	0	0	0	0	0	0	0	0	0
eventpoll	44	56	0	0	0	0	0	121306	0
spswakelock	0	0	0	0	0	0	0	0	0
bt_drv	0	0	0	0	0	0	0	0	0
wmtFuncCtrl	3	3	0	0	0	2872	1456	43697	0
MT662x	7	7	0	0	0	374	72	52259	0
vpul	0	0	0	0	0	0	0	0	0
vpul0	0	0	0	0	0	0	0	0	0
msb-hdrc	2	2	0	0	107302	134244	107302	41949	0
mdla	0	0	0	0	0	0	0	0	0
slbc	0	0	0	0	0	0	0	0	0

图 1-3. Wakelock dump 示意图

该命令需要在 UART 下输入，因为插入 USB 本身会阻止系统进入待机状态。

1.3.5 如何分析休眠功耗大的问题

- 首先看是否有频繁唤醒问题，如果有，请参考 1.3.3 章节确认唤醒源，也可以通过如下 log 关键字确认唤醒源。

```
log keywords: [SPM] suspend wake up by
```

- 如果成功进入休眠状态但功耗偏大，请先检查是否有未关闭的外设耗电。可以提供相关 log 给 Lower Power Module 进行分析定位。

```
log keywords: 26M_off_pct =
log keywords: vcore_off_pct =
```

- 如果定位到是 MediaTek SoC 造成的功耗偏大，请提供日志给 MediaTek 进行分析。抓取日志前需要开通相关 mtk kernel log，打开方式如下：

```
open log cmd: echo 1 > /proc/mtprintk
echo 1 > /proc/mtk_lpm/spm/block_threshold
```

1.3.6 如何分析 Android uos 无法 Suspend 问题

- 首先看是否有触发 Android Suspend，主要看 SoS 是否触发 IVI 进入休眠。

```
echo powerkey > /sys/guest_os/android/pm_state
```

- 如果有触发休眠，再通过 Android 端 Kernel log 确认是否有发送 Powerkey，开通 mtk kernel log 方式以及 log 关键字如下。

```
open log cmd: echo 1 > /proc/mtprintk
log keywords: [name:hyp_power_client&]host_request_suspend cmd = 0
```

- 如果确认有发送 powerkey，再看 kernel log 是否有触发 kernel suspend，关键 log 如下。

```
log keywords: suspend entry (s2idle)
```

- 如果没有触发 kernel suspend，则需要去 Check Android CPMS。
- 最后如果有触发 kernel suspend，再通过 1.3.1 中的方式判断是否成功进入 suspend。

1.3.7 Hypervisor Debug 命令

- 在 SoS 端检查 IVI (Android) OS 的状态，显示 running 表示 Android 正在运行，显示 suspend 表示 Android 已经挂起。

```
cat /sys/guest_os/android/pm_state
```

- 在 SoS 端触发 IVI (Android) OS 进入休眠状态。

```
echo powerkey > /sys/guest_os/android/pm_state
```

- 在 SoS 端触发 IVI (Android) OS 唤醒。

```
echo 0 > /sys/guest_os/android/resume
```

- 在 SoS 端检查 Tbox OS 的状态，显示 running 表示 Tbox 正在运行，显示 suspend 表示 Tbox 已经挂起。

```
cat /sys/guest_os/tbox/pm_state
```

- 在 SoS 端触发 Tbox OS 进入休眠状态。

```
echo powerkey > /sys/guest_os/tbox/pm_state
```

- 在 SoS 端触发 Tbox OS 唤醒。

```
echo 1 1 > /sys/guest_os/tbox/resume (Tbox Sleepmanager 持锁)
```

```
echo 1 2 > /sys/guest_os/tbox/resume (Tbox Sleepmanager 不持锁)
```

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。